

科沃斯机器人股份有限公司Ecovacs



科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

Linux GPIO 开发指南

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本号: 2.5
发布日期: 2023.3.6

科沃斯机器人股份有限公司Ecovacs

科沃斯机器人股份有限公司Ecovacs

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.06.29	AWA1440	添加初版。
2.0	2020.11.19	AWA1527	for linux-5.4。
2.1	2021.01.04	AWA1440	为 Linux-5.4 没有支持的接口添加警告。
2.2	2021.04.22	XAA0191	修改 Linux-5.4 中的部分描述。
2.3	2021.05.11	XAS0022	修改 Linux-5.4 中的部分描述。
2.4	2022.01.10	XAA0312	更新适用范围。
2.5	2023.3.6	XAA0249	迭代设置 debounce 的流程。

目 录

1 概述	1
1.1 编写目的	1
1.2 适用范围	1
1.3 相关人员	1
2 模块介绍	2
2.1 模块功能介绍	2
2.2 相关术语介绍	2
2.3 总体框架	3
2.4 state/pinmux/pinconfig	4
2.5 源码结构介绍	5
3 模块配置	6
3.1 kernel menuconfig 配置	6
3.2 device tree 源码结构和路径	8
3.2.1 device tree 对 gpio 控制器的通用配置	9
3.2.2 board.dts 板级配置	10
4 模块接口说明	11
4.1 pinctrl 接口说明	11
4.1.1 pinctrl_get	11
4.1.2 pinctrl_put	11
4.1.3 devm_pinctrl_get	12
4.1.4 devm_pinctrl_put	12
4.1.5 pinctrl_lookup_state	12
4.1.6 pinctrl_select_state	13
4.1.7 devm_pinctrl_get_select	13
4.1.8 devm_pinctrl_get_select_default	13
4.1.9 pin_config_get	14
4.1.10 pin_config_set	14
4.2 gpio 接口说明	14
4.2.1 gpio_request	14
4.2.2 gpio_free	15
4.2.3 gpio_direction_input	15
4.2.4 gpio_direction_output	16
4.2.5 __gpio_get_value	16
4.2.6 __gpio_set_value	16
4.2.7 of_get_named_gpio	17
4.2.8 of_get_named_gpio_flags	17

5 使用示例	18
5.1 使用 pin 的驱动 dts 配置示例	18
5.1.1 配置通用 GPIO 功能/中断功能	18
5.1.2 用法二	19
5.2 接口使用示例	20
5.2.1 配置设备引脚	20
5.2.2 获取 GPIO 号	20
5.2.3 GPIO 属性配置	21
5.3 设备驱动使用 GPIO 中断功能	23
5.4 设备驱动设置中断 debounce 功能	25
6 FAQ	28
6.1 常用 debug 方法	28
6.1.1 利用 sunxi_dump 读写相应寄存器	28
6.1.2 利用 sunxi_pinctrl 的 debug 节点	28
6.1.3 利用 pinctrl core 的 debug 节点	30
6.1.4 GPIO 中断问题排查步骤	32
6.1.4.1 GPIO 中断一直响应	32
6.1.4.2 GPIO 检测不到中断	32

插 图

图 2-1	pinctrl 驱动整体框架图	3
图 2-2	pinctrl 驱动 framework 图	4
图 3-1	内核 menuconfig 根菜单	6
图 3-2	内核 menuconfig device drivers 菜单	7
图 3-3	内核 menuconfig pinctrl drivers 菜单	7
图 3-4	内核 menuconfig allwinner pinctrl drivers 菜单	8
图 6-1	查看 pin 配置图	29
图 6-2	修改结果图	29
图 6-3	pin 设备图	30
图 6-4	pin 设备图	30

1 概述

1.1 编写目的

本文档对内核的 GPIO 接口使用进行详细的阐述，让用户明确掌握 GPIO 配置、申请等操作的编程方法。

1.2 适用范围

表 1-1: 适用产品列表

内核版本	驱动文件
Linux-4.9	{SDK}/kernel/linux-4.9/drivers/pinctrl/sunxi
Linux-5.4	{SDK}/kernel/linux-5.4/drivers/pinctrl/sunxi
Linux-5.10 Linux-5.15	{SDK}/bsp/drivers/pinctrl

1.3 相关人员

本文档适用于所有需要在 Linux 内核 sunxi 平台上开发设备驱动的相关人员。

2 模块介绍

Pinctrl 框架是 linux 系统为统一各 SoC 厂商 pin 管理，避免各 SoC 厂商各自实现相同 pin 管理子系统而提出的。目的是为了减少 SoC 厂商系统移植工作量。

2.1 模块功能介绍

许多 SoC 内部都包含 pin 控制器，通过 pin 控制器，我们可以配置一个或一组引脚的功能和特性。在软件上，Linux 内核 pinctrl 驱动可以操作 pin 控制器为我们完成如下工作：

- 枚举并且命名 pin 控制器可控制的所有引脚；
- 提供引脚的复用能力
- 提供配置引脚的能力，如驱动能力、上拉下拉、数据属性等。
- 与 gpio 子系统的交互
- 实现 pin 中断

2.2 相关术语介绍

表 2-1: Pinctrl 模块相关术语介绍

术语	解释说明
SUNXI	Allwinner 一系列 SOC 硬件平台。
Pin controller	是对硬件模块的软件抽象，通常用来表示硬件控制器。能够处理引脚复用、属性配置等功能。
Pin	根据芯片不同的封装方式，可以表现为球形、针型等。软件上采用常用一组无符号的整数 [0-maxpin] 来表示。
Pin groups	外围设备通常都不只一个引脚，比如 SPI，假设接在 SoC 的 {0,8,16,24} 管脚，而另一个设备 I2C 接在 SoC 的 {24,25} 管脚。我们可以说这里有两个 pin groups。很多控制器都需要处理 pin groups。因此管脚控制器子系统需要一个机制用来枚举管脚组且检索一个特定组中实际枚举的管脚。
Pinconfig	管脚可以被软件配置成多种方式，多数与它们作为输入/输出时的电气特性相关。例如，可以设置一个输出管脚处于高阻状态，或是“三态”（意味着它被有效地断开连接）。或者可以通过设置将一个输入管脚与 VDD 或 GND 相连（上拉/下拉），以便在没有信号驱动管脚时使管脚拥有确定值。

术语	解释说明
Pinmux	引脚复用功能，使用一个特定的物理管脚（ball/pad/finger/等等）进行多种扩展复用，以支持不同功能的电气封装习惯。
Device tree	犹如它的名字，是一棵包括 cpu 的数量和类别、内存基地址、总线与桥、外设连接，中断控制器和 gpio 以及 clock 等系统资源的树，Pinctrl 驱动支持从 device tree 中定义的设备节点获取 pin 的配置信息。
SDK	指的是 longan 或者 Tina SDK 的根目录。

2.3 总体框架

Sunxi Pinctrl 驱动模块的框架如下图所示，整个驱动模块可以分成 4 个部分：pinctrl api、pinctrl common frame、sunxi pinctrl driver，以及 board configuration。（图中最上面一层 device driver 表示 Pinctrl 驱动的使用者）

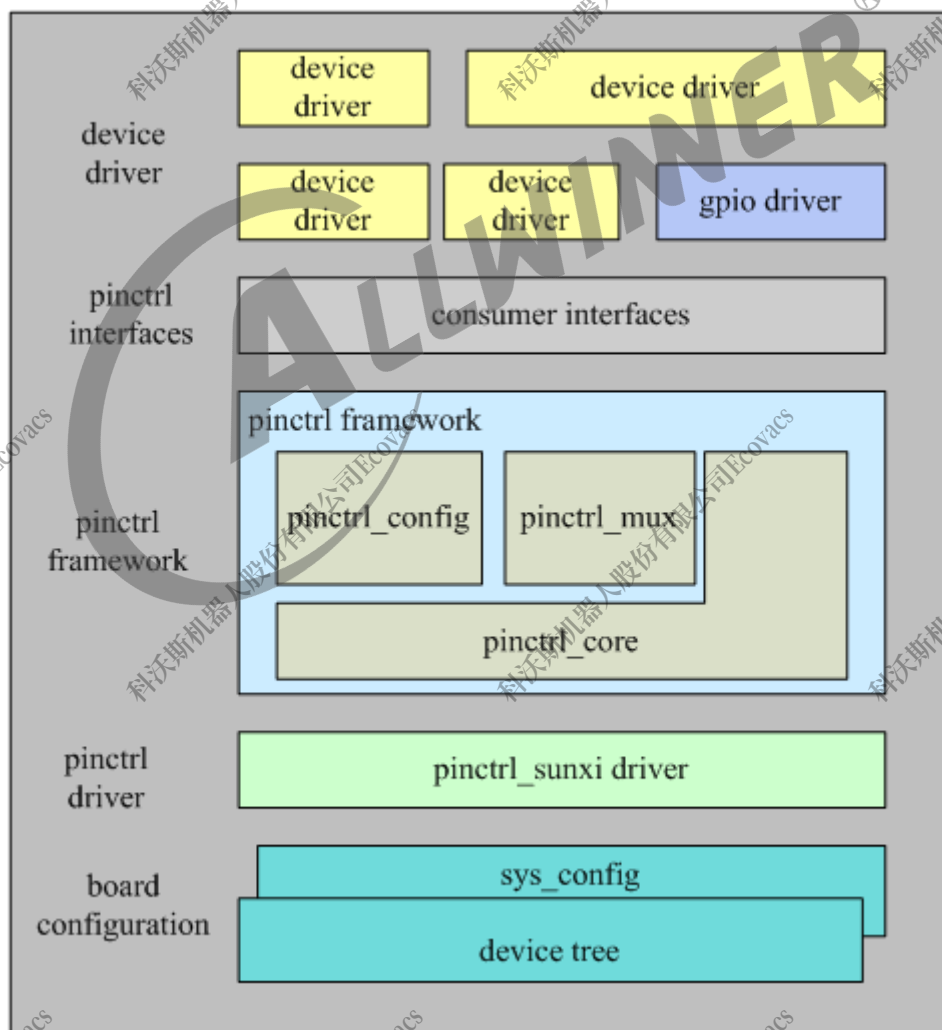


图 2-1: pinctrl 驱动整体框架图

Pinctrl api: pinctrl 提供给上层用户调用的接口。

Pinctrl framework: Linux 提供的 pinctrl 驱动框架。

Pinctrl sunxi driver: sunxi 平台需要实现的驱动。

Board configuration: 设备 pin 配置信息，一般采用设备树进行配置。

2.4 state/pinmux/pinconfig

Pinctrl framework 主要处理 pinstate、pinmux 和 pinconfig 三个功能，pinstate 和 pinmux、pinconfig 映射关系如下图所示。

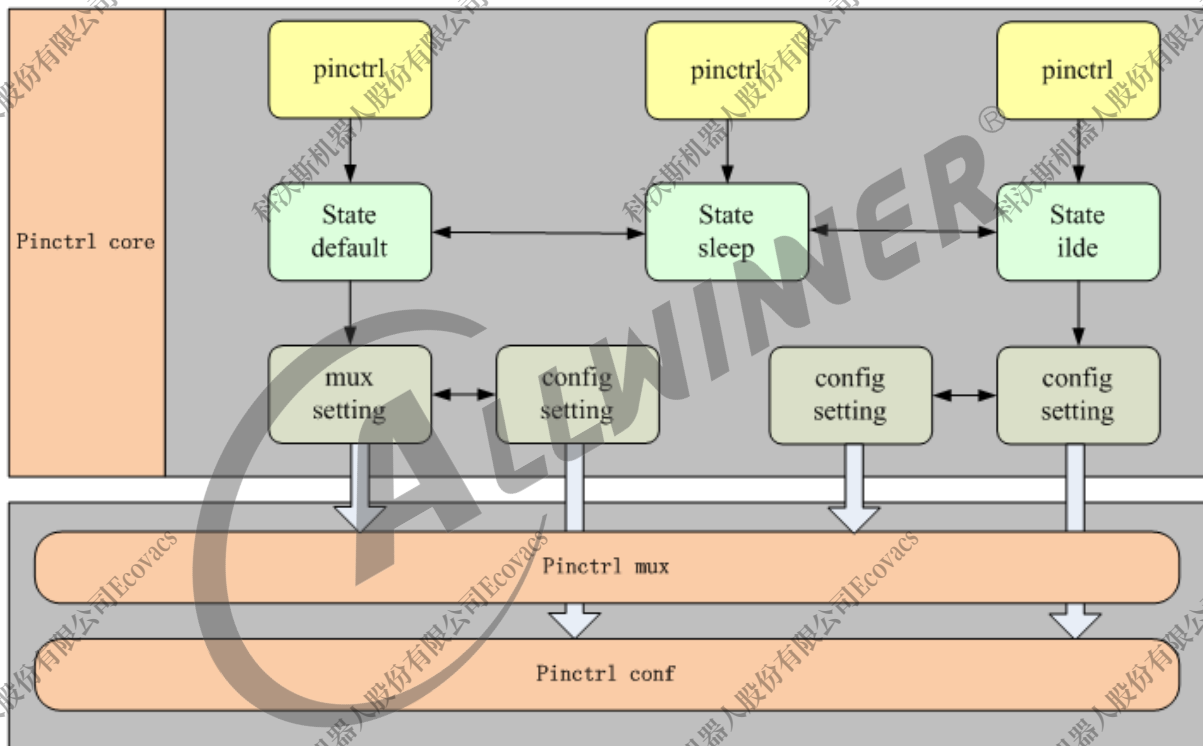


图 2-2: pinctrl 驱动framework 图

系统运行在不同的状态，pin 配置有可能不一样，比如系统正常运行时，设备的 pin 需要一组配置，但系统进入休眠时，为了节省功耗，设备 pin 需要另一组配置。Pinctrl framwork 能够有效管理设备在不同状态下的引脚配置。

2.5 源码结构介绍

```
linux
|-- drivers
|   |-- pinctrl
|   |   |-- Kconfig
|   |   |-- Makefile
|   |   |-- core.c
|   |   |-- core.h
|   |   |-- devicetree.c
|   |   |-- devicetree.h
|   |   |-- pinconf.c
|   |   |-- pinconf.h
|   |   |-- pinmux.c
|   |   |-- pinmux.h
|   |-- sunxi
|   |   |-- pinctrl-sunxi-test.c
|   |   |-- pinctrl-sun*.c
|   |   |-- pinctrl-sun*-r.c
|-- include
|   |-- linux
|   |   |-- pinctrl
|   |   |   |-- consumer.h
|   |   |   |-- devinfo.h
|   |   |   |-- machine.h
|   |   |   |-- pinconf-generic.h
|   |   |   |-- pinconf.h
|   |   |   |-- pinctrl-state.h
|   |   |   |-- pinctrl.h
|   |   |   |-- pinmux.h
```

3 模块配置

3.1 kernel menuconfig 配置

进入 {SDK} 根目录，执行 `./build.sh menuconfig`

进入配置主界面，并按以下步骤操作：

首先，选择 Device Drivers 选项进入下一级配置，如下图所示：

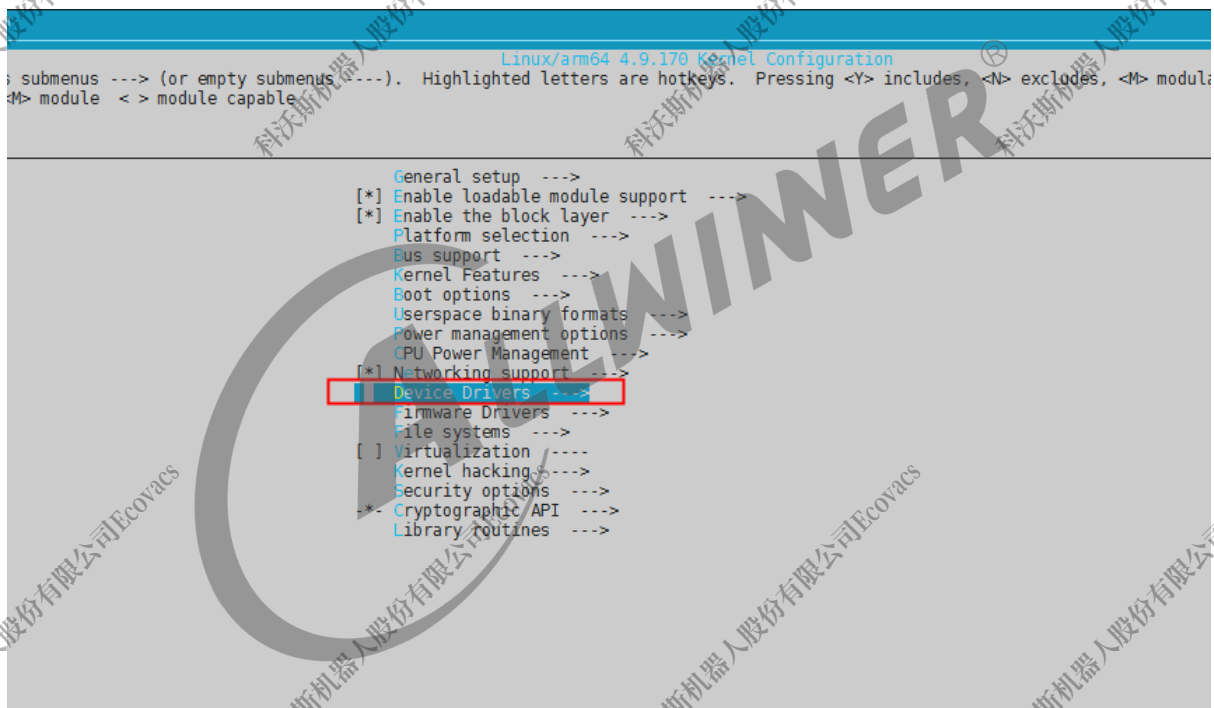


图 3-1: 内核 menuconfig 根菜单

选择 Pin controllers, 进入下级配置，如下图所示：

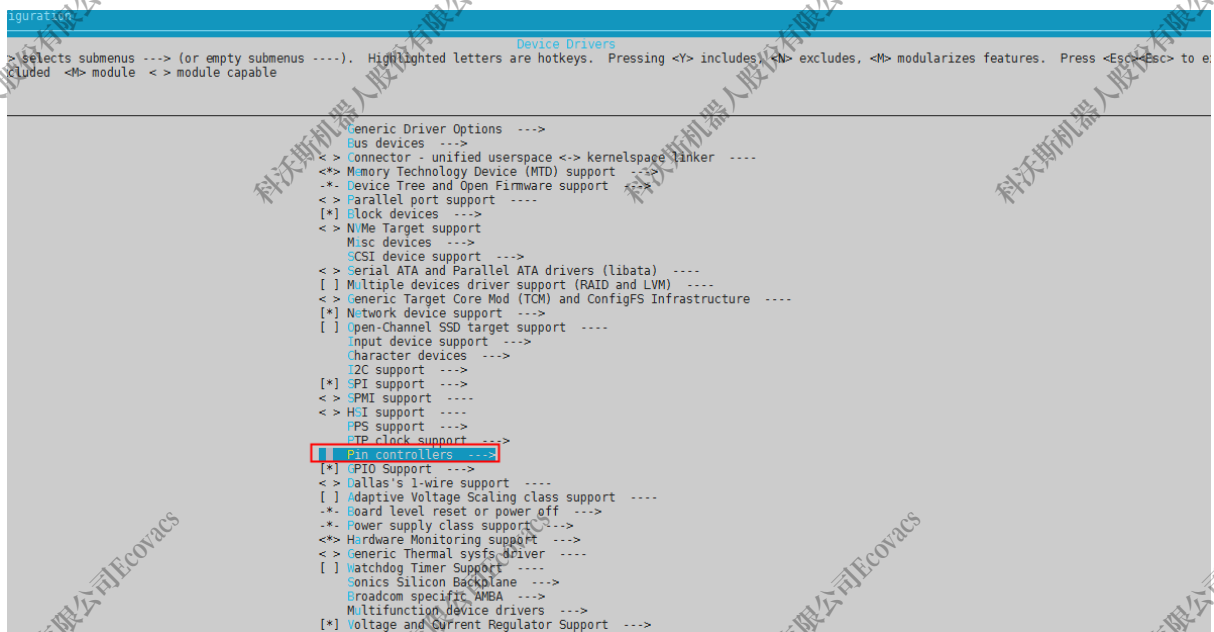


图 3-2: 内核 menuconfig device drivers 菜单

选择 Allwinner SoC PINCTRL DRIVER, 进入下级配置, 如下图所示:

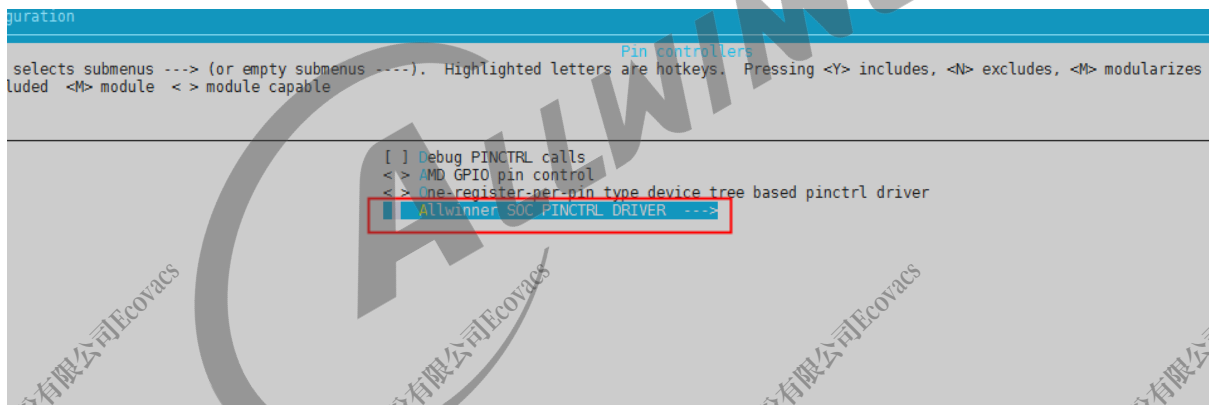


图 3-3: 内核 menuconfig pinctrl drivers 菜单

Sunxi pinctrl driver 默认编译进内核, 如下图 (以 sun50iw9p1 平台为例, 其他平台类似) 所示:

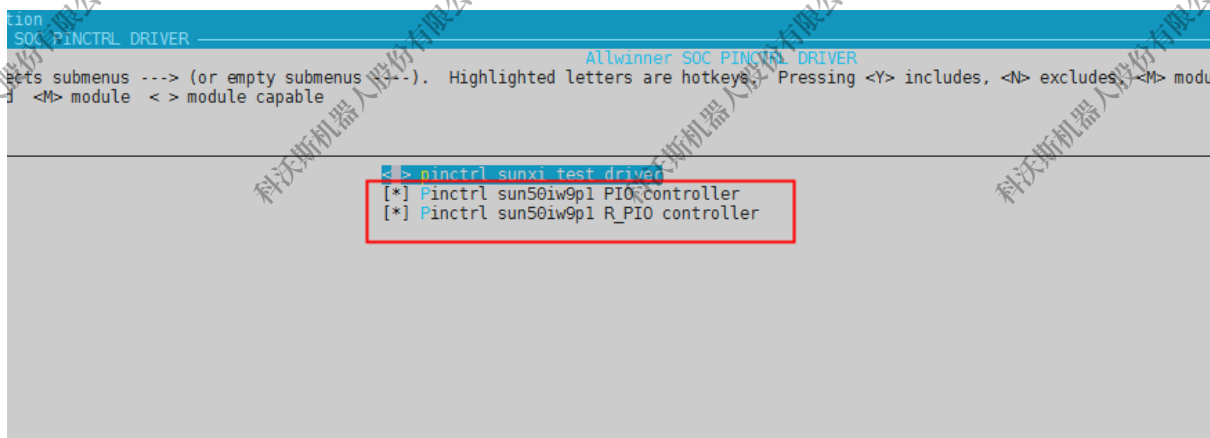


图 3-4: 内核 menuconfig allwinner pinctrl drivers 菜单

3.2 device tree 源码结构和路径

对于 Linux-4.9:

- 设备树文件的配置是该 SoC 所有方案的通用配置，对于 ARM64 CPU 而言，设备树的路径为：kernel/{KERNEL}/arch/arm64/boot/dts/sunxi/sun*-pinctrl.dtsi。
- 设备树文件的配置是该 SoC 所有方案的通用配置，对于 ARM32 CPU 而言，设备树的路径为：kernel/{KERNEL}/arch/arm32/boot/dts/sun*-pinctrl.dtsi。
- 板级设备树 (board.dts) 路径：/device/config/chips/{IC}/configs/{BOARD}/board.dts

device tree 的源码结构关系如下：

```
board.dts
|-----sun*.dtsi
|-----sun*-pinctrl.dtsi
|-----sun*-clk.dtsi
```

对于 Linux-5.4:

- 设备树文件的配置是该 SoC 所有方案的通用配置，对于 ARM64 CPU 而言，5.4 内核中不再维护单独的 pinctrl 的 dtsi，直接将 pin 的信息放在了：kernel/{KERNEL}/arch/arm32/boot/dts/sun*.dtsi
- 设备树文件的配置是该 SoC 所有方案的通用配置，对于 ARM32 CPU 而言，5.4 内核中不再维护单独的 pinctrl 的 dtsi，直接将 pin 的信息放在了：kernel/{KERNEL}/arch/arm32/boot/dts/sun*.dtsi
- 板级设备树 (board.dts) 路径：/device/config/chips/{IC}/configs/{BOARD}/board.dts
- device tree 的源码包含关系如下：

```
board.dts
|-----sun*.dtsi
```

3.2.1 device tree 对 gpio 控制器的通用配置

在 kernel/{KERNEL}/arch/arm64/boot/dts/sunxi/sun-pinctrl.dtsi 文件中 (Linux-5.4 直接放在 sun.dtsi 中)，配置了该 SoC 的 pinctrl 控制器的通用配置信息，一般不建议修改，有 pinctrl 驱动维护者维护。目前，在 sunxi 平台，我们根据电源域，注册两个 pinctrl 设备：r_pio 设备 (PL0 后的所有 pin) 和 pio 设备 (PL0 前的所有 pin)，两个设备的通用配置信息如下：

```

1  r_pio: pinctrl@07022000 {
2      compatible = "allwinner,sun50iw9p1-r-pinctrl"; //兼容属性，用于驱动和设备绑定
3      reg = <0x0 0x07022000 0x0 0x400>; //寄存器基地址0x07022000和范围0x400
4      clocks = <&clk_cpurio>; //r_pio设置使用的时钟
5      device_type = "r_pio"; //设备类型属性
6      gpio-controller; //表示是一个gpio控制器
7      interrupt-controller; //表示一个中断控制器，不支持中断可以删除
8      #interrupt-cells = <3>; //pin中断属性需要配置的参数个数，不支持中断可以删除
9      #size-cells = <0>; //没有使用，配置0
10     #gpio-cells = <6>; //gpio属性配置需要的参数个数,对于linux-5.4为3
11
12     /*
13     * 以下配置为模块使用的pin的配置，模块通过引用相应的节点对pin进行操作
14     * 由于不同板级的pin经常改变，建议通过板级dts修改（参考下一小节）
15     */
16     s_rsb0_pins_a: s_rsb0@0 {
17         allwinner,pins = "PL0", "PL1";
18         allwinner,function = "s_rsb0";
19         allwinner,muxsel = <2>;
20         allwinner,drive = <2>;
21         allwinner,pull = <1>;
22     };
23
24     /*
25     * 以下配置为linux-5.4模块使用pin的配置，模块通过引用相应的节点对pin进行操作
26     * 由于不同板级的pin经常改变，建议将模块pin的引用放到board dts中
27     * (类似pinctrl-0 = <&scr1_ph_pins>;),并使用scr1_ph_pins这种更有标识性的名字。
28     */
29     scr1_ph_pins: scr1-ph-pins {
30         pins = "PH0", "PH1";
31         function = "sim1";
32         drive-strength = <10>;
33         bias-pull-up;
34     };
35 };
36
37 pio: pinctrl@0300b000 {
38     compatible = "allwinner,sun50iw9p1-pinctrl"; //兼容属性，用于驱动和设备绑定
39     reg = <0x0 0x0300b000 0x0 0x400>; //寄存器基地址0x0300b000和范围0x400
40     interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>, /* AW1823_GIC_Spec: GPIOA: 83-32=51 */
41                 <GIC_SPI 52 IRQ_TYPE_LEVEL_HIGH>,
42                 <GIC_SPI 53 IRQ_TYPE_LEVEL_HIGH>,
43                 <GIC_SPI 54 IRQ_TYPE_LEVEL_HIGH>,
44                 <GIC_SPI 55 IRQ_TYPE_LEVEL_HIGH>,
45                 <GIC_SPI 56 IRQ_TYPE_LEVEL_HIGH>,
46                 <GIC_SPI 57 IRQ_TYPE_LEVEL_HIGH>; //该设备每个bank支持的中断配置和gic中断号，每个中断号对应
47     一个支持中断的bank
48     device_type = "pio"; //设备类型属性
49     clocks = <&clk_pio>, <&clk_losc>, <&clk_hosc>; //该设备使用的时钟
50     gpio-controller; //表示是一个gpio控制器

```

```
50 interrupt-controller; //表示是一个中断控制器
51 #interrupt-cells = <3>; //pin中断属性需要配置的参数个数，不支持中断可以删除
52 #size-cells = <0>; //没有使用
53 #gpio-cells = <6>; //gpio属性需要配置的参数个数，对于linux-5.4为3
54 /* takes the debounce time in usec as argument */
55 }
```

3.2.2 board.dts 板级配置

board.dts 用于保存每个板级平台的设备信息 (如 demo 板、demo2.0 板等等)，以 demo 板为例，board.dts 路径如下：

/device/config/chips/{CHIP}/configs/demo/board.dts

在 board.dts 中的配置信息如果在 *.dtsi 中 (如 sun50iw9p1.dtsi 等) 存在，则会存在以下覆盖规则：

- 相同属性和结点，board.dts 的配置信息会覆盖 *.dtsi 中的配置信息。
- 新增加的属性和结点，会追加到最终生成的 dtb 文件中。

linux-4.9 上面 pinctrl 中一些模块使用 board.dts 的简单配置如下：

```
1 pio: pinctrl@0300b000 {
2     input-debounce = <0 0 0 0 0 0>; /* 配置中断采样频率，每个对应一个支持中断的bank，单位us*/
3
4     spi0_pins_a: spi0@0 {
5         allwinner,pins = "PC0", "PC2", "PC4";
6         allwinner,pname = "spi0_sclk", "spi0_mosi", "spi0_miso";
7         allwinner,function = "spi0";
8     };
9 }
```

对于 linux-5.4，不建议采用上面的覆盖方式，而是修改驱动 pinctrl-0 引用的节点。

linux-5.4 上面 board.dts 的配置如下：

```
1 &pio{
2     input-debounce = <0 0 0 0 1 0 0 0>; //配置中断采样频率，每个对应一个支持中断的bank，单位us
3     vcc-pe-supply = <&reg_pio1_8>; //配置IO口耐压值，例如这里的含义是将pe口设置成1.8v耐压值
4 };
```

4 模块接口说明

4.1 pinctrl 接口说明

4.1.1 pinctrl_get

- 函数原型：struct pinctrl *pinctrl_get(struct device *dev);
- 作用：获取设备的 pin 操作句柄，所有 pin 操作必须基于此 pinctrl 句柄。
- 参数：
 - dev: 指向申请 pin 操作句柄的设备句柄。
- 返回：
 - 成功，返回 pinctrl 句柄。
 - 失败，返回 NULL。

4.1.2 pinctrl_put

- 函数原型：void pinctrl_put(struct pinctrl *p)
- 作用：释放 pinctrl 句柄，必须与 pinctrl_get 配对使用。
- 参数：
 - p: 指向释放的 pinctrl 句柄。
- 返回：
 - 没有返回值。

⚠ 注意

必须与 pinctrl_get 配对使用。

4.1.3 devm_pinctrl_get

- 函数原型：struct pinctrl *devm_pinctrl_get(struct device *dev)
- 作用：根据设备获取 pin 操作句柄，所有 pin 操作必须基于此 pinctrl 句柄，与 pinctrl_get 功能完全一样，只是 devm_pinctrl_get 会将申请到的 pinctrl 句柄做记录，绑定到设备句柄信息中。设备驱动申请 pin 资源，推荐优先使用 devm_pinctrl_get 接口。
- 参数：
 - dev: 指向申请 pin 操作句柄的设备句柄。
- 返回：
 - 成功，返回 pinctrl 句柄。
 - 失败，返回 NULL。

4.1.4 devm_pinctrl_put

- 函数原型：void devm_pinctrl_put(struct pinctrl *p)
- 作用：释放 pinctrl 句柄，必须与 devm_pinctrl_get 配对使用。
- 参数：
 - p: 指向释放的 pinctrl 句柄。
- 返回：
 - 没有返回值。



注意

必须与 devm_pinctrl_get 配对使用，可以不显式的调用该接口。

4.1.5 pinctrl_lookup_state

- 函数原型：struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name)
- 作用：根据 pin 操作句柄，查找 state 状态句柄。
- 参数：
 - p: 指向要操作的 pinctrl 句柄。
 - name: 指向状态名称，如 “default”、“sleep” 等。
- 返回：
 - 成功，返回执行 pin 状态的句柄 struct pinctrl_state *。
 - 失败，返回 NULL。

4.1.6 pinctrl_select_state

- 函数原型：int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *s)
- 作用：将 pin 句柄对应的 pinctrl 设置为 state 句柄对应的状态。
- 参数：
 - p: 指向要操作的 pinctrl 句柄。
 - s: 指向 state 句柄。
- 返回：
 - 成功，返回 0。
 - 失败，返回错误码。

4.1.7 devm_pinctrl_get_select

- 函数原型：struct pinctrl *devm_pinctrl_get_select(struct device *dev, const char *name)
- 作用：获取设备的 pin 操作句柄，并将句柄设定为指定状态。
- 参数：
 - dev: 指向管理 pin 操作句柄的设备句柄。
 - name: 要设置的 state 名称，如 “default”、“sleep” 等。
- 返回：
 - 成功，返回 pinctrl 句柄。
 - 失败，返回 NULL。

4.1.8 devm_pinctrl_get_select_default

- 函数原型：struct pinctrl *devm_pinctrl_get_select_default(struct device *dev)
- 作用：获取设备的 pin 操作句柄，并将句柄设定为默认状态。
- 参数：
 - dev: 指向管理 pin 操作句柄的设备句柄。
- 返回：
 - 成功，返回 pinctrl 句柄。
 - 失败，返回 NULL。

4.1.9 pin_config_get

- 作用：获取指定 pin 的属性。
- 参数：
 - dev_name: 指向 pinctrl 设备。
 - name: 指向 pin 名称。
 - config: 保存 pin 的配置信息。
- 返回：
 - 成功，返回 pin 编号。
 - 失败，返回错误码。

⚠ 注意

该接口在 linux-5.4 已经移除。

4.1.10 pin_config_set

- 作用：设置指定 pin 的属性。
- 参数：
 - dev_name: 指向 pinctrl 设备。
 - name: 指向 pin 名称。
 - config: pin 的配置信息。
- 返回：
 - 成功，返回 0。
 - 失败，返回错误码。

⚠ 注意

该接口在 linux-5.4 已经移除。

4.2 gpio 接口说明

4.2.1 gpio_request

- 函数原型：int gpio_request(unsigned gpio, const char *label)

- 作用：申请 gpio，获取 gpio 的访问权。
- 参数：
 - gpio:gpio 编号。
 - label:gpio 名称，可以为 NULL。
- 返回：
 - 成功，返回 0。
 - 失败，返回错误码。

4.2.2 gpio_free

- 函数原型：void gpio_free(unsigned gpio)
- 作用：释放 gpio。
- 参数：
 - gpio:gpio 编号。
- 返回：
 - 无返回值。

4.2.3 gpio_direction_input

- 函数原型：int gpio_direction_input(unsigned gpio)
- 作用：设置 gpio 为 input。
- 参数：
 - gpio:gpio 编号。
- 返回：
 - 成功，返回 0。
 - 失败，返回错误码。

4.2.4 gpio_direction_output

- 函数原型：int gpio_direction_output(unsigned gpio, int value)
- 作用：设置 gpio 为 output。
- 参数：
 - gpio:gpio 编号。
 - value: 期望设置的 gpio 电平值，非 0 表示高, 0 表示低。
- 返回：
 - 成功，返回 0。
 - 失败，返回错误码。

4.2.5 __gpio_get_value

- 函数原型：int __gpio_get_value(unsigned gpio)
- 作用：获取 gpio 电平值 (gpio 已为 input/output 状态)。
- 参数：
 - gpio:gpio 编号。
- 返回：
 - 返回 gpio 对应的电平逻辑，1 表示高, 0 表示低。

4.2.6 __gpio_set_value

- 函数原型：void __gpio_set_value(unsigned gpio, int value)
- 作用：设置 gpio 电平值 (gpio 已为 input/output 状态)。
- 参数：
 - gpio:gpio 编号。
 - value: 期望设置的 gpio 电平值，非 0 表示高, 0 表示低。
- 返回：
 - 无返回值

4.2.7 of_get_named_gpio

- 函数原型：int of_get_named_gpio(struct device_node *np, const char *propname, int index)
- 作用：通过名称从 dts 解析 gpio 属性并返回 gpio 编号。
- 参数：
 - np: 指向使用 gpio 的设备结点。
 - propname: dts 中属性的名称。
 - index: dts 中属性的索引值。
- 返回：
 - 成功，返回 gpio 编号。
 - 失败，返回错误码。

4.2.8 of_get_named_gpio_flags

- 函数原型：int of_get_named_gpio_flags(struct device_node *np, const char *list_name, int index, enum of_gpio_flags * flags)
- 作用：通过名称从 dts 解析 gpio 属性并返回 gpio 编号。
- 参数：
 - np: 指向使用 gpio 的设备结点。
 - propname: dts 中属性的名称。
 - index: dts 中属性的索引值
 - flags: 在 sunxi 平台上，必须定义为 struct gpio_config * 类型变量，因为 sunxi pinctrl 的 pin 支持上下拉，驱动能力等信息，而内核 enum of_gpio_flags * 类型变量只能包含输入、输出信息，后续 sunxi 平台需要标准化该接口。
- 返回：
 - 成功，返回 gpio 编号。
 - 失败，返回错误码。

⚠ 注意

该接口的 flags 参数，在 sunxi linux-4.9 及以前的平台上，必须定义为 struct gpio_config 类型变量。linux-5.4 已经标准化该接口，直接采用 enum of_gpio_flags 的定义。

5 使用示例

5.1 使用 pin 的驱动 dts 配置示例

对于使用 pin 的驱动来说，驱动主要设置 pin 的常用的几种功能，列举如下：

- 驱动使用者只配置通用 GPIO, 即用来做输入、输出和中断的
- 驱动使用者设置 pin 的 pin mux, 如 uart 设备的 pin, lcd 设备的 pin 等, 用于特殊功能
- 驱动使用者既要配置 pin 的通用功能, 也要配置 pin 的特性

下面对常见使用场景进行分别介绍。

5.1.1 配置通用 GPIO 功能/中断功能

用法一：配置 GPIO，中断，device tree 配置 demo 如下所示：

```
1 soc{
2     ...
3     gpiokey {
4         device_type = "gpiokey";
5         compatible = "gpio-keys";
6
7         ok_key {
8             device_type = "ok_key";
9             label = "ok_key";
10            gpios = <&r_pio PL 0x4 0x0 0x1 0x0 0x1>; // 如果是linux 3.4, 则应该为 gpios = <&r_pio 0 4
11            GPIO_ACTIVE_HIGH>;
12            linux, input-type = "1>";
13            linux, code = <0x1c>;
14            wakeup-source = <0x1>;
15        };
16    };
17    ...
18};
```



```
gpios = <&r_pio PL 0x4 0x0 0x1 0x0 0x1>;
```

使用上述方式配置gpio时，需要驱动调用以下接口解析dts的配置参数：

拿到gpio的配置信息后(保存在flags参数中, 见4.2.8.小节), 在根据需要调用相应的标准接口实现自己的功能

```
gpios = < &r_pio 0 4 GPIO_ACTIVE_HIGH >;
```

5.1.2 用法二

用法二：配置设备引脚，device tree 配置 demo 如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
28     pinctrl-names = "default", "sleep";
29     pinctrl-0 = <&uart0_pins_a>;
30     pinctrl-1 = <&uart0_pins_b>;
31     uart0_regulator = "vcc-id";
32     uart0_port = <0>;
33     uart0_type = <2>;
34 };
35 ...
36 ;
```

其中：

- pinctrl-0 对应 pinctrl-names 中的 default，即模块正常工作模式下对应的 pin 配置。
- pinctrl-1 对应 pinctrl-names 中的 sleep，即模块休眠模式下对应的 pin 配置。

5.2 接口使用示例

5.2.1 配置设备引脚

一般设备驱动只需要使用一个接口 devm_pinctrl_get_select_default 就可以申请到设备所有 pin 资源。

```
1 static int sunxi_pin_req_demo(struct platform_device *pdev)
2 {
3     struct pinctrl *pinctrl;
4
5     /* request device pinctrl, set as default state */
6     pinctrl = devm_pinctrl_get_select_default(&pdev->dev);
7     if (IS_ERR_OR_NULL(pinctrl))
8         return -EINVAL;
9
10    return 0;
11 }
```

5.2.2 获取 GPIO 号

```
1 static int sunxi_pin_req_demo(struct platform_device *pdev)
2 {
3     struct device *dev = &pdev->dev;
4     struct device_node *np = dev->of_node;
5     unsigned int gpio;
6
7     #get gpio config in device node.
8     gpio = of_get_named_gpio(np, "vdevice_3", 0);
9     if (!gpio_is_valid(gpio)) {
10         if (gpio != -EPROBE_DEFER)
11             dev_err(dev, "Error getting vdevice_3\n");
12     }
13 }
```

```

12     return gpio;
13 }
14 }

```

5.2.3 GPIO 属性配置

通过 pin_config_set/pin_config_get/pin_config_group_set/pin_config_group_get 接口单独控制指定 pin 或 group 的相关属性。

```

1 static int pctrltest_request_all_resource(void)
2 {
3     struct device *dev;
4     struct device_node *node;
5     struct pinctrl *pinctrl;
6     struct sunxi_gpio_config *gpio_list = NULL;
7     struct sunxi_gpio_config *gpio_cfg;
8     unsigned gpio_count = 0;
9     unsigned gpio_index;
10    unsigned long config;
11    int ret;
12
13    dev = bus_find_device_by_name(&platform_bus_type, NULL, sunxi_ptest_data->dev_name);
14    if (!dev) {
15        pr_warn("find device [%s] failed...\n", sunxi_ptest_data->dev_name);
16        return -EINVAL;
17    }
18
19    node = of_find_node_by_type(NULL, dev_name(dev));
20    if (!node) {
21        pr_warn("find node for device [%s] failed...\n", dev_name(dev));
22        return -EINVAL;
23    }
24    dev->of_node = node;
25
26    pr_warn("+++++++++++++++++++++%s++++++++++++++++++++\n", __func__);
27    pr_warn("device[%s] all pin resource we want to request\n", dev_name(dev));
28    pr_warn("-----\n");
29
30    pr_warn("step1: request pin all resource.\n");
31    pinctrl = devm_pinctrl_get_select_default(dev);
32    if (IS_ERR_OR_NULL(pinctrl)) {
33        pr_warn("request pinctrl handle for device [%s] failed...\n", dev_name(dev));
34        return -EINVAL;
35    }
36
37
38    pr_warn("step2: get device[%s] pin count.\n", dev_name(dev));
39    ret = dt_get_gpio_list(node, &gpio_list, &gpio_count);
40    if (ret < 0 || gpio_count == 0) {
41        pr_warn(" devices own 0 pin resource or look for main key failed!\n");
42        return -EINVAL;
43    }
44
45    pr_warn("step3: get device[%s] pin configure and check.\n", dev_name(dev));
46    for (gpio_index = 0; gpio_index < gpio_count; gpio_index++) {

```

```

47     gpio_cfg = &gpio_list[gpio_index];
48
49     /* check function config */
50     config = SUNXI_PINCFG_PACK(SUNXI_PINCFG_TYPE_FUNC, 0xFFFF);
51     pin_config_get(SUNXI_PINCTRL, gpio_cfg->name, &config);
52     if (gpio_cfg->mulsel != SUNXI_PINCFG_UNPACK_VALUE(config)) {
53         pr_warn("failed! mul value isn't equal as dt.\n");
54         return -EINVAL;
55     }
56
57     /* check pull config */
58     if (gpio_cfg->pull != GPIO_PULL_DEFAULT) {
59         config = SUNXI_PINCFG_PACK(SUNXI_PINCFG_TYPE_PUD, 0xFFFF);
60         pin_config_get(SUNXI_PINCTRL, gpio_cfg->name, &config);
61         if (gpio_cfg->pull != SUNXI_PINCFG_UNPACK_VALUE(config)) {
62             pr_warn("failed! pull value isn't equal as dt.\n");
63             return -EINVAL;
64         }
65     }
66
67     /* check dlevel config */
68     if (gpio_cfg->drive != GPIO_DRVLVL_DEFAULT) {
69         config = SUNXI_PINCFG_PACK(SUNXI_PINCFG_TYPE_DRV, 0xFFFF);
70         pin_config_get(SUNXI_PINCTRL, gpio_cfg->name, &config);
71         if (gpio_cfg->drive != SUNXI_PINCFG_UNPACK_VALUE(config)) {
72             pr_warn("failed! dlevel value isn't equal as dt.\n");
73             return -EINVAL;
74         }
75     }
76
77     /* check data config */
78     if (gpio_cfg->data != GPIO_DATA_DEFAULT) {
79         config = SUNXI_PINCFG_PACK(SUNXI_PINCFG_TYPE_DAT, 0xFFFF);
80         pin_config_get(SUNXI_PINCTRL, gpio_cfg->name, &config);
81         if (gpio_cfg->data != SUNXI_PINCFG_UNPACK_VALUE(config)) {
82             pr_warn("failed! pin data value isn't equal as dt.\n");
83             return -EINVAL;
84         }
85     }
86 }
87
88 pr_warn("-----\n");
89 pr_warn("test pinctrl request all resource success!\n");
90 pr_warn("++++++end++++++\n");
91 return 0;
92 }
93

```

注：需要注意，存在SUNXI_PINCTRL和SUNXI_R_PINCTRL两个pinctrl设备，cpus域的pin需要使用SUNXI_R_PINCTRL

⚠ 注意

linux-5.4 中使用 `pinctrl_gpio_set_config` 配置 `gpio` 属性，对应使用 `pinconf_to_config_pack` 生成 `config` 参数：

- `SUNXI_PINCFG_TYPE_FUNC` 已不再生效，暂未支持 `FUNC` 配置（建议使用 `pinctrl_select_state` 接口代替）。
- `SUNXI_PINCFG_TYPE_PUD` 更新为内核标准定义（`PIN_CONFIG_BIAS_PULL_UP/PIN_CONFIG_BIAS_PULL_DOWN`）。
- `SUNXI_PINCFG_TYPE_DRV` 更新为内核标准定义（`PIN_CONFIG_DRIVE_STRENGTH`），相应的 `val` 对应关系为（4.9->5.4: 0->10, 1->20...）。
- `SUNXI_PINCFG_TYPE_DAT` 已不再生效，暂未支持 `DAT` 配置（建议使用 `gpio_direction_output` 或者 `__gpio_set_value` 设置电平值）。

5.3 设备驱动使用 GPIO 中断功能

方式一：通过 `gpio_to_irq` 获取虚拟中断号，然后调用申请中断函数即可。

目前 `sunxi-pinctrl` 使用 `irq-domain` 为 `gpio` 中断实现虚拟 `irq` 的功能，使用 `gpio` 中断功能时，设备驱动只需要通过 `gpio_to_irq` 获取虚拟中断号后，其他均可以按标准 `irq` 接口操作。

```
1 static int sunxi_gpio_eint_demo(struct platform_device *pdev)
2 {
3     struct device *dev = &pdev->dev;
4     int virq;
5     int ret;
6     /* map the virq of gpio */
7     virq = gpio_to_irq(GPIOA(0));
8     if (IS_ERR_VALUE(virq)) {
9         pr_warn("map gpio [%d] to virq failed, errno = %d\n",
10             GPIOA(0), virq);
11         return -EINVAL;
12     }
13     pr_debug("gpio [%d] map to virq [%d] ok\n", GPIOA(0), virq);
14     /* request virq, set virq type to high level trigger */
15     ret = devm_request_irq(dev, virq, sunxi_gpio_irq_test_handler,
16         IRQF_TRIGGER_HIGH, "PA0_EINT", NULL);
17     if (IS_ERR_VALUE(ret)) {
18         pr_warn("request virq %d failed, errno = %d\n", virq, ret);
19         return -EINVAL;
20     }
21     return 0;
22 }
```

方式二：通过 `dtb` 配置 `gpio` 中断，通过 `dtb` 解析函数获取虚拟中断号，最后调用申请中断函数即可，demo 如下所示：

```
1 dtb配置如下：
2 soc{
3     ...
4     Vdevice: vdevice@0 {
5         compatible = "allwinner,sun8i-vdevice";
6         device_type = "Vdevice";
7     }
```

```

7  interrupt-parent = <&pio>; /* 依赖的中断控制器(带interrupt-controller属性的结点) */
8  interrupts = < PD 3 IRQ_TYPE_LEVEL_HIGH>;
9      | | ----- 中断触发条件、类型
10     | | ----- pin bank内偏移
11     | | ----- 哪个bank
12  pinctrl-names = "default";
13  pinctrl-0 = <&vdevice_pins_a>;
14  test-gpios = <&pio PC 3 1 2 2 1>;
15  status = "okay";
16  };
17  ...
18  };

```

在驱动中，通过 platform_get_irq() 标准接口获取虚拟中断号，如下所示：

```

1  static int sunxi_pctrltest_probe(struct platform_device *pdev)
2  {
3      struct device_node *np = pdev->dev.of_node;
4      struct gpio_config config;
5      int gpio, irq;
6      int ret;
7
8      if (np == NULL) {
9          pr_err("Vdevice failed to get of_node\n");
10         return -ENODEV;
11     }
12     ....
13     irq = platform_get_irq(pdev, 0);
14     if (irq < 0) {
15         printk("Get irq error!\n");
16         return -EBUSY;
17     }
18     .....
19     sunxi_ptest_data->irq = irq;
20     .....
21     return ret;
22 }
23
24 // 申请中断:
25 static int pctrltest_request_irq(void)
26 {
27     int ret;
28     int virq = sunxi_ptest_data->irq;
29     int trigger = IRQF_TRIGGER_HIGH;
30
31     reinit_completion(&sunxi_ptest_data->done);
32
33     pr_warn("step1: request irq(%s level) for irq:%d.\n",
34           trigger == IRQF_TRIGGER_HIGH ? "high" : "low", virq);
35     ret = request_irq(virq, sunxi_pinctrl_irq_handler_demo1,
36                     trigger, "PIN_EINT", NULL);
37     if (IS_ERR_VALUE(ret)) {
38         pr_warn("request irq failed !\n");
39         return -EINVAL;
40     }
41
42     pr_warn("step2: wait for irq.\n");
43     ret = wait_for_completion_timeout(&sunxi_ptest_data->done, HZ);
44     if (ret == 0) {

```

```

45     pr_warn("wait for irq timeout!\n");
46     free_irq(virq, NULL);
47     return -EINVAL;
48 }
49
50 free_irq(virq, NULL);
51
52 pr_warn("-----\n");
53 pr_warn("test pin eint success !\n");
54 pr_warn("++++++end++++++\n\n\n");
55
56 return 0;
57 }

```

5.4 设备驱动设置中断 debounce 功能

理论中断采样频率最大 24M Hz，最小 4K Hz，但由于不同内核版本、不同配置方式的不同，有不同的取值范围。

方式一：通过 dts 配置每个中断 bank 的 debounce，以 pio 设备为例，如下所示：

```

1 &pio {
2     /* takes the debounce time in usec as argument */
3     input-debounce = <0 0 0 0 0 0>;
4
5     PA bank
6     PC bank
7     PD bank
8     PF bank
9     PG bank
10    PH bank
11    PI bank
};

```

注意：* input-debounce 的属性值中需把 pio 设备支持中断的 bank 都配上，对于平台没有的 bank 或不支持中断的 bank，跳过不写。* 如果缺少，会以 bank 的顺序设置相应的属性值到 debounce 寄存器，缺少的 bank 对应的 debounce 应该是默认值（启动时没修改的情况）。

sunxi linux-4.9 平台

- 由于寄存器限制和运算限制，中断采样频率最大可配置 750K，最小 32k，可配数值如下所示，一般填 1 可以满足需求。

若是填入不属于以下表格的数据，会自动计算到最接近的一组数据上去。

dts 数值	实际采样时间 (us)	采样频率 (HZ)
0	31.25	32K
1	1.33	750K

dts 数值	实际采样时间 (us)	采样频率 (HZ)
2	2.67	375K
5	5.33	187.5K

linux-5.4 * debounce 取值范围是 0~1000000（单位 usec）。

方式二：驱动模块调用 gpio 相关接口设置中断 debounce。

在驱动中，调用下面两个接口即可设置 gpio 对应的中断 debounce 寄存器 (linux-5.4 已经移除这个接口)。

函数 1：

- 函数原型：static inline int gpio_set_debounce(unsigned gpio, unsigned debounce)
- 作用：设置 gpio 对应的中断 debounce 寄存器。
- 参数：
 - gpio:gpio 编号。
 - debounce: 要给 debounce 寄存器设置的寄存器数值
- 返回：
 - 返回设置结果，0 表示成功，负值代表失败。

函数 2：

- 函数原型：int gpiod_set_debounce(struct gpio_desc *desc, unsigned debounce)
- 作用：设置 gpio 对应的中断 debounce 寄存器。
- 参数：
 - desc: gpio 的 desc 结构体指针。
 - debounce: 要给 debounce 寄存器设置的寄存器数值
- 返回：
 - 返回设置结果，0 表示成功，负值代表失败。

寄存器数值与实际采样频率对应关系如下

寄存器数值	实际采样时间 (us)	采样频率 (HZ)
0x01	0.04	24000000
0x11	0.08	12000000
0x21	0.17	6000000

寄存器数值	实际采样时间 (us)	采样频率 (HZ)
0x31	0.33	3000000
0x41	0.67	1500000
0x51	1.33	750000
0x61	2.67	375000
0x71	5.33	187500
0x00	31.25	32000
0x10	62.50	16000
0x20	125.00	8000
0x30	250.00	4000
0x40	500.00	2000
0x50	1000.00	1000
0x60	2000.00	500
0x70	4000.00	250

6 FAQ

6.1 常用 debug 方法

6.1.1 利用 sunxi_dump 读写相应寄存器

需要开启 SUNXI_DUMP 模块：

```
make kernel_menuconfig  
---> Device Drivers  
---> dump reg driver for sunxi platform (选中)
```

使用方法：

```
1 cd /sys/class/sunxi_dump  
2 1. 查看一个寄存器  
3   echo 0x0300b048 > dump ; cat dump  
4  
5 2. 写值到寄存器上  
6   echo 0x0300b058 0xff > write ; cat write  
7  
8 3. 查看一片连续寄存器  
9   echo 0x0300b000,0x0300bfff > dump ; cat dump  
10  
11 4. 写一组寄存器的值  
12   echo 0x0300b058 0xff,0x0300b0a0 0xff > write ; cat write  
13  
14 通过上述方式，可以查看，修改相应gpio的寄存器，从而发现问题所在。
```

6.1.2 利用 sunxi_pinctrl 的 debug 节点

需要开启 DEBUG_FS：

```
make kernel_menuconfig  
---> Kernel hacking  
---> Compile-time checks and compiler options  
---> Debug Filesystem (选中)
```

挂载文件节点，并进入相应目录：

```
1 mount -t debugfs none /sys/kernel/debug
2
3 cd /sys/kernel/debug/sunxi_pinctrl
```

1. 查看 pin 的配置:

```
1 echo PC2 > sunxi_pin
2 cat sunxi_pin_configure
```

结果如下图所示:

```
/sys/kernel/debug # cd sunxi_pinctrl/
/sys/kernel/debug/sunxi_pinctrl # ls
data                function            sunxi_pin
device              platform           sunxi_pin_configure
dlevel              pull
/sys/kernel/debug/sunxi_pinctrl # echo PC2 > sunxi_pin
/sys/kernel/debug/sunxi_pinctrl # cat sunxi_pin_configure
pin[PC2] funciton: 4
pin[PC2] data: 0
pin[PC2] dlevel: 1
pin[PC2] pull: 0
```

图 6-1: 查看 pin 配置图

2. 修改 pin 属性

每个 pin 都有四种属性, 如复用 (function), 数据 (data), 驱动能力 (dlevel), 上下拉 (pull), 修改 pin 属性的命令如下:

```
1 echo PC2 1 > pull; cat pull
2 cat sunxi_pin_configure // 查看修改情况
```

修改后结果如下图所示:

```
/sys/kernel/debug/sunxi_pinctrl # echo PC2 > sunxi_pin
/sys/kernel/debug/sunxi_pinctrl # cat sunxi_pin_configure
pin[PC2] funciton: 4
pin[PC2] data: 0
pin[PC2] dlevel: 1
pin[PC2] pull: 0
/sys/kernel/debug/sunxi_pinctrl # echo PC2 1 > pull
/sys/kernel/debug/sunxi_pinctrl # cat sunxi_pin_configure
pin[PC2] funciton: 4
pin[PC2] data: 0
pin[PC2] dlevel: 1
pin[PC2] pull: 1
```

图 6-2: 修改结果图

注意：在 sunxi 平台，目前多个 pinctrl 的设备，分别是 pio 和 r_pio 和 apxxxx-gpio，当操作 PL 之后的 pin 时，请通过以下命令切换 pin 的设备，否则操作失败，切换命令如下：

```
1 echo pio > /sys/kernel/debug/sunxi_pinctrl/dev_name //切换到pio设备
2 cat /sys/kernel/debug/sunxi_pinctrl/dev_name
3 echo r_pio > /sys/kernel/debug/sunxi_pinctrl/dev_name //切换到r_pio设备
4 cat /sys/kernel/debug/sunxi_pinctrl/dev_name
```

修改结果如下图所示：

```
/sys/kernel/debug/sunxi_pinctrl # echo r_pio > dev_name ;cat dev_name
r_pio
/sys/kernel/debug/sunxi_pinctrl # echo pio > dev_name ;cat dev_name
pio
/sys/kernel/debug/sunxi_pinctrl #
```

图 6-3: pin 设备图

6.1.3 利用 pinctrl core 的 debug 节点

```
1 mount -t debugfs none /sys/kernel/debug
2
3 cd /sys/kernel/debug/sunxi_pinctrl
```

1. 查看 pin 的管理设备：

```
1 cat pinctrl-devices
```

结果如下图所示：

```
130|console:/sys/kernel/debug/pinctrl # ls
pinctrl-devices pinctrl-handles pinctrl-maps pio r_pio
console:/sys/kernel/debug/pinctrl # cat pinctrl-devices
name [pinmux] [pinconf]
r_pio yes yes
pio yes yes
console:/sys/kernel/debug/pinctrl #
```

图 6-4: pin 设备图

2. 查看 pin 的状态和对应的使用设备

```
1 cat pinctrl-handles
```

结果如下图 log 所示：

```
console:/sys/kernel/debug/pinctrl # ls
pinctrl-devices pinctrl-handles pinctrl-maps pio r_pio
console:/sys/kernel/debug/pinctrl # cat pinctrl-handles
Requested pin control handlers their pinmux maps:
device: twi3 current state: sleep
state: default
type: MUX_GROUP controller pio group: PA10 (10) function: twi3 (15)
type: CONFIGS_GROUP controller pio group PA10 (10)config 00001409
config 00000005
type: MUX_GROUP controller pio group: PA11 (11) function: twi3 (15)
type: CONFIGS_GROUP controller pio group PA11 (11)config 00001409
config 00000005
state: sleep
type: MUX_GROUP controller pio group: PA10 (10) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PA10 (10)config 00001409
config 00000001
type: MUX_GROUP controller pio group: PA11 (11) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PA11 (11)config 00001409
config 00000001
device: twi5 current state: default
state: default
type: MUX_GROUP controller r_pio group: PL0 (0) function: s_twi0 (3)
type: CONFIGS_GROUP controller r_pio group PL0 (0)config 00001409
config 00000005
type: MUX_GROUP controller r_pio group: PL1 (1) function: s_twi0 (3)
type: CONFIGS_GROUP controller r_pio group PL1 (1)config 00001409
config 00000005
state: sleep
type: MUX_GROUP controller r_pio group: PL0 (0) function: io_disabled (4)
type: CONFIGS_GROUP controller r_pio group PL0 (0)config 00001409
config 00000001
type: MUX_GROUP controller r_pio group: PL1 (1) function: io_disabled (4)
type: CONFIGS_GROUP controller r_pio group PL1 (1)config 00001409
config 00000001
device: soc@03000000:pwm5@0300a000 current state: active
state: active
type: MUX_GROUP controller pio group: PA12 (12) function: pwm5 (16)
type: CONFIGS_GROUP controller pio group PA12 (12)config 00000001
config 00000000
config 00000000
state: sleep
type: MUX_GROUP controller pio group: PA12 (12) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PA12 (12)config 00000001
config 00000000
config 00000000
device: uart0 current state: default
state: default
state: sleep
device: uart1 current state: default
state: default
type: MUX_GROUP controller pio group: PG6 (95) function: uart1 (37)
type: CONFIGS_GROUP controller pio group PG6 (95)config 00001409
config 00000005
type: MUX_GROUP controller pio group: PG7 (96) function: uart1 (37)
type: CONFIGS_GROUP controller pio group PG7 (96)config 00001409
config 00000005
type: MUX_GROUP controller pio group: PG8 (97) function: uart1 (37)
type: CONFIGS_GROUP controller pio group PG8 (97)config 00001409
config 00000005
type: MUX_GROUP controller pio group: PG9 (98) function: uart1 (37)
```

```
type: CONFIGS_GROUP controller pio group PG9 (98)config 00001409
config 00000005
state: sleep
type: MUX_GROUP controller pio group: PG6 (95) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PG6 (95)config 00001409
config 00000001
type: MUX_GROUP controller pio group: PG7 (96) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PG7 (96)config 00001409
config 00000001
type: MUX_GROUP controller pio group: PG8 (97) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PG8 (97)config 00001409
config 00000001
type: MUX_GROUP controller pio group: PG9 (98) function: io_disabled (5)
type: CONFIGS_GROUP controller pio group PG9 (98)config 00001409
....
```

从上面的部分 log 可以看到那些设备管理的 pin 以及 pin 当前的状态是否正确。以 twi3 设备为例，twi3 管理的 pin 有 PA10/PA11，分别有两组状态 sleep 和 default，default 状态表示使用状态，sleep 状态表示 pin 处于 io disabled 状态，表示 pin 不可正常使用，twi3 设备使用的 pin 当前状态处于 sleep 状态的。

6.1.4 GPIO 中断问题排查步骤

6.1.4.1 GPIO 中断一直响应

1. 排查中断信号是否一直触发中断。
2. 利用 sunxi_dump 节点，确认中断 pending 位是否没有清 (参考 6.1.1 小节)。
3. 是否在 gpio 中断服务程序里对中断检测的 gpio 进行 pin mux 的切换，不允许这样切换，否则会导致中断异常。

6.1.4.2 GPIO 检测不到中断

1. 排查中断信号是否正常，若不正常，则排查硬件，若正常，则跳到步骤 2。
2. 利用 sunxi_dump 节点，查看 gpio 中断 pending 位是否置起，若已经置起，则跳到步骤 5，否则跳到步骤 3。
3. 利用 sunxi_dump 节点，查看 gpio 的中断触发方式是否配置正确，若正确，则跳到步骤 4，否则跳到步骤 5。
4. 检查中断的采样时钟，默认应该是 32k，可以通过 sunxi_dump 节点，切换 gpio 中断采样时钟到 24M 进行实验。
5. 利用 sunxi_dump，确认中断是否使能。

著作权声明

版权所有 © 2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明



（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。